شرح سطر بسطر ٤ — المكوّنات المشتركة (core/widgets)

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

هذه ودجتات تتكرر في أكثر من شاشة، فكتبناها مرة واحدة: الزر الرئيسي، مربع الأيقونة، شريط التقدّم، الشريط السفلي، الشخصية الملوّنة، الشريط العلوي وزر الرجوع، المفتاح، البطاقة، الحد المنقّط، ونقاط/لوحة الرمز السري.

القاعدة العامة: كل ودجت بلا حالة داخلية (`StatelessWidget`) ويستقبل ما يحتاجه كمعاملات، فيسهل اختباره وإعادة استعماله.

تذكير بالاتجاه: في RTL يظهر أول عنصر في `Row` على اليمين؛ لذلك ترتيب العناصر في الكود هو ترتيب القراءة العربي.

lib/core/widgets/icon_tile.dart — مربع الأيقونة

`lib/core/widgets/icon_tile.dart`

الأسطر 1–4

```
1  import 'package:flutter/material.dart';
2  import 'package:flutter_svg/flutter_svg.dart';
3
4  import 'app_icon.dart';
```

ماذا يفعل؟ الاستيرادات: Material وflutter_svg و`AppIcon`.

لماذا بهذه الطريقة؟ —

الأسطر 6–9

```
6  /// مربع ملوّن بزوايا مدوّرة يحوي أيقونة في الوسط.
7  /// الأيقونة إما ملف SVG من Figma ([asset])، أو أيقونة Material ([iconData])
8  /// للأيقونات التي لم تُصدَّر من Figma بعد.
9  class IconTile extends StatelessWidget {
```

ماذا يفعل؟ تعليق الصنف: مربع ملوّن بأيقونة في الوسط من ثلاثة مصادر ممكنة.

لماذا بهذه الطريقة؟ —

الأسطر 10–19

```
  const IconTile({
    super.key,
    this.asset,
    this.iconData,
    this.iconName,
    this.iconColor,
    required this.color,
    this.size = 48,
    this.iconSize = 24,
    this.radius = 14,
```

ماذا يفعل؟ المنشئ: الأيقونة من `asset` (ملف SVG ملوّن) أو `iconData` (أيقونة Material) أو `iconName` (أيقونة من مجموعة التصميم)، ولونها، ولون المربع والمقاسات.

لماذا بهذه الطريقة؟ ثلاثة مصادر لأن بعض الأيقونات مصدّرة ملوّنة، وبعضها من مجموعة التصميم أحادية اللون تُصبغ، وأيقونات Material احتياط.

الأسطر 20–26

```
  }) : assert(
         (asset != null ? 1 : 0) +
                 (iconData != null ? 1 : 0) +
                 (iconName != null ? 1 : 0) ==
             1,
         'مرّر واحدًا فقط من asset أو iconData أو iconName',
       );
```

ماذا يفعل؟ `assert` يتأكد أن مصدرًا واحدًا بالضبط مُرّر.

لماذا بهذه الطريقة؟ يكشف خطأ البرمجة في debug بدل أيقونة فارغة صامتة.

الأسطر 28–37

```
  final String? asset;
  final IconData? iconData;

  /// اسم أيقونة من مجموعة التصميم (انظر [AppIcon])، تُلوَّن بـ [iconColor].
  final String? iconName;
  final Color? iconColor;
  final Color color;
  final double size;
  final double iconSize;
  final double radius;
```

ماذا يفعل؟ الحقول، و`iconName` يُلوَّن بـ `iconColor`.

لماذا بهذه الطريقة؟ —

الأسطر 39–48

```
39    @override
40    Widget build(BuildContext context) {
41      return Container(
42        width: size,
43        height: size,
44        decoration: BoxDecoration(
45          color: color,
46          borderRadius: BorderRadius.circular(radius),
47        ),
48        alignment: Alignment.center,
```

ماذا يفعل؟ البناء: حاوية بالمقاس واللون والزوايا، والمحتوى في الوسط.

لماذا بهذه الطريقة؟ —

الأسطر 49–60

```
49        child: switch ((asset, iconName)) {
50          (final a?, _) => SvgPicture.asset(a, width: iconSize, height: iconSize),
51          (_, final n?) => AppIcon(
52            n,
53            size: iconSize,
54            color: iconColor ?? Colors.black,
55          ),
56          _ => Icon(iconData, size: iconSize, color: iconColor),
57        },
58      );
59    }
60  }
```

ماذا يفعل؟ اختيار المصدر بنمط السجلات: SVG كما هو، أو `AppIcon` بالاسم واللون (أسود افتراضيًا)، أو Material `Icon`. ثم الإغلاق.

لماذا بهذه الطريقة؟ `switch` على السجل `(asset, iconName)` أقصر من سلسلة `if`، والحالة الأخيرة تعني `iconData`.

lib/core/widgets/progress_bar.dart — شريط التقدّم

lib/core/widgets/progress_bar.dart

الأسطر 1–4

```
1   import 'package:flutter/material.dart';
2
3   /// شريط تقدّم يبدأ من اليمين (اتجاه القراءة العربي).
4   class ProgressBar extends StatelessWidget {
```

ماذا يفعل؟ استيراد Material وتعليق وصنف `ProgressBar`.

لماذا بهذه الطريقة؟ يبدأ من اليمين (اتجاه القراءة العربي).

الأسطر 5–11

```
  const ProgressBar({
    super.key,
    required this.value,
    required this.trackColor,
    required this.fillColor,
    this.height = 6,
  }) : assert(value >= 0 && value <= 1);
```

ماذا يفعل؟ الباني: `value` ولونا المسار والتعبئة مطلوبة، و`height` افتراضيه 6. و`assert` أن القيمة بين 0 و1.

لماذا بهذه الطريقة؟ يلتقط الخطأ في التطوير (مثل تمرير 75 بدل 0.75). التصميم يستعمل نفس الشريط بارتفاعين وألوان.

الأسطر 13–17

```
  /// بين 0 و 1.
  final double value;
  final Color trackColor;
  final Color fillColor;
  final double height;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ `value` بين 0 و1 ليتوافق مع `widthFactor` مباشرة.

الأسطر 19–21

```
  @override
  Widget build(BuildContext context) {
    final radius = BorderRadius.circular(height / 2);
```

ماذا يفعل؟ `build` ويحسب نصف قطر الزوايا = نصف الارتفاع.

لماذا بهذه الطريقة؟ يعطي أطرافًا دائرية تمامًا (شكل كبسولة) بأي ارتفاع.

الأسطر 23–26

```
    return Container(
      height: height,
      decoration: BoxDecoration(color: trackColor, borderRadius: radius),
      alignment: AlignmentDirectional.centerStart,
```

ماذا يفعل؟ `Container` للمسار: ارتفاع ولون ونصف قطر، ومحاذاة `AlignmentDirectional.centerStart`.

لماذا بهذه الطريقة؟ أهم سطر: `centerStart` = اليمين في RTL فتبدأ التعبئة من اليمين كما في `centerLeft`. Figma كان سيعكس الشكل.

الأسطر 27–33

```
        child: FractionallySizedBox(
          widthFactor: value,
          heightFactor: 1,
          child: DecoratedBox(
            decoration: BoxDecoration(color: fillColor, borderRadius: radius),
          ),
        ),
```

ماذا يفعل؟ `FractionallySizedBox` بعرض `value` من عرض المسار، فيه مستطيل بلون التعبئة.

لماذا بهذه الطريقة؟ يحسب العرض نسبيًا تلقائيًا بلا `LayoutBuilder` ولا معرفة العرض بالبكسل.

الأسطر 34–36

```
      );
    }
  }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف.

lib/core/widgets/bottom_nav.dart — الشريط السفلي

`lib/core/widgets/bottom_nav.dart`

السلاسة: كل تبويب داخل `Flexible` (يتقلّص عند الضيق)، ويُلحَق بـ `.pressable()` ، وعند اختيار تبويب جديد (لا الحالي) يهتزّ الجهاز نقرة خفيفة `Haptics.tap()` .

الأسطر 1–9

```
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';

import '../constants/app_assets.dart';
import '../strings/ar.dart';
import '../theme/app_colors.dart';
import '../theme/app_text_styles.dart';
import 'pressable_scale.dart';
import '../utils/haptics.dart';
```

ماذا يفعل؟ استيراد Material وSVG والأصول والنصوص والألوان والأنماط.

لماذا بهذه الطريقة؟ ما يحتاجه الشريط.

السطر 11

```
enum RizmaTab { home, subjects, progress, profile }
```

ماذا يفعل؟ تعداد `RizmaTab` بأربع قيم.

لماذا بهذه الطريقة؟ نمرّر التبويب الحالي قيمةً نوعية آمنة بدل رقم فيستحيل خلط ترتيبها.

الأسطر 13–15

```
/// الشريط السفلي: الرئيسية · المواد · تقدّمي · حسابي (من اليمين لليسار).
class BottomNav extends StatelessWidget {
  const BottomNav({super.key, required this.current, required this.onSelected});
```

ماذا يفعل؟ تعليق وصنف `BottomNav` والباني بحقلين مطلوبين.

لماذا بهذه الطريقة؟ الأولى يمينًا: الرئيسية ثم المواد ثم تقدّمي ثم حسابي.

الأسطر 17–18

```
  final RizmaTab current;
  final ValueChanged<RizmaTab> onSelected;
```

ماذا يفعل؟ الحقلان: التبويب الحالي ودالة الاختيار.

لماذا بهذه الطريقة؟ الشريط لا يحتفظ بحالة؛ من يستعمله يقرر الحالي ويمرّره.

الأسطر 20–37

```
  static const _items = [
    (tab: RizmaTab.home, label: ArStrings.navHome, asset: AppAssets.navHome),
    (
      tab: RizmaTab.subjects,
      label: ArStrings.navSubjects,
      asset: AppAssets.navBook,
    ),
    (
      tab: RizmaTab.progress,
      label: ArStrings.navProgress,
      asset: AppAssets.navChart,
    ),
    (
      tab: RizmaTab.profile,
      label: ArStrings.navProfile,
      asset: AppAssets.navUser,
    ),
  ];
```

ماذا يفعل؟ قائمة ثابتة من أربعة سجلات (record): التبويب والاسم والأيقونة.

لماذا بهذه الطريقة؟ سجلات Dart 3 المسمّاة أخف من صنف جديد لبيانات صغيرة ومحلية. ترتيب القائمة هو ترتيب Figma.

الأسطر 39–40

```
  @override
  Widget build(BuildContext context) {
```

ماذا يفعل؟ `build`.

لماذا بهذه الطريقة؟ يعيد الشريط.

الأسطر 41–45

```
    return DecoratedBox(
      decoration: const BoxDecoration(
        color: AppColors.card,
        border: Border(top: BorderSide(color: AppColors.line)),
      ),
```

ماذا يفعل؟ `DecoratedBox` بخلفية بيضاء وحدّ علوي رفيع.

لماذا بهذه الطريقة؟ الحد يفصل الشريط عن المحتوى كما في Figma.

الأسطر 46–47

```
      child: SafeArea(
        top: false,
```

ماذا يفعل؟ `SafeArea` من الأسفل فقط.

لماذا بهذه الطريقة؟ تحمي الشريط من شريط الإيماءات أسفل الهاتف.

الأسطر 48–49

```
        child: Padding(
          padding: const EdgeInsetsDirectional.fromSTEB(16, 10, 16, 18),
```

ماذا يفعل؟ حشوة 16 أفقيًا و10 أعلى و18 أسفل.

لماذا بهذه الطريقة؟ قيم Figma (بصيغة `Directional`).

الأسطر 50–52

```
          child: Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
```

ماذا يفعل؟ `Row` بتوزيع `spaceBetween` .

لماذا بهذه الطريقة؟ يوزّع التبويبات الأربعة بالتساوي بين الحافتين.

الأسطر 53–65

```
              // Flexible: على شاشة ضيقة أو بخط مكبّر تنكمش العناصر بدل أن تفيض.
              for (final item in _items)
                Flexible(
                  child: _NavItem(
                    label: item.label,
                    asset: item.asset,
                    selected: item.tab == current,
                    onTap: () {
                      if (item.tab != current) Haptics.tap();
                      onSelected(item.tab);
                    },
                  ),
                ),
```

ماذا يفعل؟ تعليق ثم حلقة تبني `_NavItem` لكل عنصر داخل `Flexible`؛ المحدَّد هو من يساوي `current`، والضغط يستدعي `onSelected`.

لماذا بهذه الطريقة؟ `Flexible` يجعل العنصر ينكمش عند الضيق: على شاشة 320 وخط مكبّر ×1.5 كان الصف يفيض بـ 63 بكسلًا (اكتشفه اختبار `layout_stress_test`). مع `spaceBetween` يبقى التوزيع نفسه على الشاشات العادية.

الأسطر 66–72

```
            ],
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية `BottomNav`.

الأسطر 74–80

```
class _NavItem extends StatelessWidget {
  const _NavItem({
    required this.label,
    required this.asset,
    required this.selected,
    required this.onTap,
  });
```

ماذا يفعل؟ ودجت `_NavItem` خاص وباني بأربعة معاملات.

لماذا بهذه الطريقة؟ عنصر واحد من الشريط.

الأسطر 82–85

```
82    final String label;
83    final String asset;
84    final bool selected;
85    final VoidCallback onTap;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 87–89

```
87    @override
88    Widget build(BuildContext context) {
89      final color = selected ? AppColors.teal : AppColors.soft;
```

ماذا يفعل؟ `build` ويحدد اللون: teal للمحدَّد وإلا `soft` .

لماذا بهذه الطريقة؟ لون الأيقونة والنص من متغيّر واحد فلا يختلفان.

الأسطر 91–95

```
91      return Semantics(
92        button: true,
93        selected: selected,
94        label: label,
95        excludeSemantics: true,
```

ماذا يفعل؟ `Semantics` : زر ومحدَّد أم لا واسمه.

لماذا بهذه الطريقة؟ قارئ الشاشة يقول «الرئيسية، محدَّد».

الأسطر 96–99

```
96        child: Material(
97          color: selected ? AppColors.tealTint : Colors.transparent,
98          borderRadius: BorderRadius.circular(16),
99          clipBehavior: Clip.antiAlias,
```

ماذا يفعل؟ `Material` بخلفية `tealTint` للمحدَّد وشفافة لغيره، وزوايا 16 وقصّ.

لماذا بهذه الطريقة؟ هذه «الكبسولة الخضراء» مؤشر التبويب الحالي في Figma.

الأسطر 100–101

```
100         child: InkWell(
101           onTap: onTap,
```

ماذا يفعل؟ `InkWell` ينفّذ `onTap` .

لماذا بهذه الطريقة؟ الضغط على أي مكان داخل العنصر.

الأسطر 102–103

```
          child: Padding(
            padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
```

ماذا يفعل؟ حشوة 12 أفقيًا و6 عموديًا.

لماذا بهذه الطريقة؟ من Figma.

الأسطر 104–107

```
            child: Column(
              mainAxisSize: MainAxisSize.min,
              spacing: 2,
              children: [
```

ماذا يفعل؟ `Column` بحجم أدنى ومسافة 2: أيقونة فوق النص.

لماذا بهذه الطريقة؟ `min` ليحيط العنصر بمحتواه فقط.

الأسطر 108–113

```
                SvgPicture.asset(
                  asset,
                  width: 22,
                  height: 22,
                  colorFilter: ColorFilter.mode(color, BlendMode.srcIn),
                ),
```

ماذا يفعل؟ الأيقونة بحجم 22 ومرشّح لوني `ColorFilter.mode(color, srcIn)`.

لماذا بهذه الطريقة؟ أيقونات التنقّل خطوط بلون واحد؛ `srcIn` يصبغها بلون التبويب فنستعمل ملفًا واحدًا لكل أيقونة بدل ملفين.

الأسطر 114–121

```
                FittedBox(
                  fit: BoxFit.scaleDown,
                  child: Text(
                    label,
                    maxLines: 1,
                    style: AppTextStyles.captionBold13.copyWith(color: color),
                  ),
                ),
```

ماذا يفعل؟ نص الاسم بنمط `captionBold13` ولون التبويب، داخل `FittedBox(scaleDown)` وسطر واحد.

لماذا بهذه الطريقة؟ وزن 700 ليبرز بحجم صغير. `FittedBox` يصغّر النص إن لم يتسع بدل قصّه أو الفيض، ولا يغيّره في الحالة العادية.

الأسطر 122–129

```
              ],
            ),
          ),
        ),
      ),
    ).pressable();
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/core/widgets/mascot.dart — الشخصية الملوّنة

lib/core/widgets/mascot.dart

استُبدلت التميمة القديمة بتميمة Figma (دفتر بشريط أحمر وورقتين) في ثلاث حالات: مبتسمة، نائمة، مفكّرة. تُلوَّن بثلاثة ألوان من لوحة الطالب.

الأسطر 1–6

```
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';

import '../constants/app_assets.dart';
import '../db/enums.dart';
import '../theme/profile_palette.dart';
```

ماذا يفعل؟ استيراد Material و`flutter_svg` والأصول والتعداد ولوحة ألوان الملف.

لماذا بهذه الطريقة؟ نحتاج `ColorMapper` من `flutter_svg` لتلوين الرسم وقت العرض.

الأسطر 8–18

```
/// حالات التميمة في التصميم الجديد.
enum MascotMood {
  /// مبتسمة (الافتراضي).
  happy,

  /// نائمة: «قيد التحضير»، أو لا شيء الآن.
  sleepy,

  /// تفكّر (علامة استفهام): التلميحات والرمز الخاطئ
  thinking,
}
```

ماذا يفعل؟ `MascotMood`: مبتسمة (الافتراضي)، نائمة («قيد التحضير» أو لا شيء الآن)، مفكّرة (التلميحات والرمز الخاطئ).

لماذا بهذه الطريقة؟ التصميم يغيّر وجه التميمة حسب الموقف؛ تعداد يمنع قيمًا عشوائية.

الأسطر 20–33

```dart
/// تميمة رزمة (دفتر بشريط أحمر وورقتين) بلون الطالب. الأصل أخضر، فنستبدل
/// ألوانه الثلاثة (الجسم والغامق والورقتين) وقت العرض بدل الاحتفاظ بخمسة ملفات.
/// اللوحة 150×150 نفسها التي تُرسم عليها إكسسوارات الخزانة.
class Mascot extends StatelessWidget {
  const Mascot({
    super.key,
    required this.size,
    this.color = ProfileColor.teal,
    this.mood = MascotMood.happy,
  });

  final double size;
  final ProfileColor color;
  final MascotMood mood;
```

ماذا يفعل؟ تعليق وصنف `Mascot`: الحجم مطلوب، واللون (أخضر افتراضيًّا) والمزاج اختياريان.

لماذا بهذه الطريقة؟ الأصل أخضر؛ نستبدل ألوانه الثلاثة وقت العرض بدل الاحتفاظ بخمسة عشر ملفًا (خمسة ألوان × ثلاث حالات).

الأسطر 35–52

```dart
  @override
  Widget build(BuildContext context) {
    return SvgPicture.asset(
      switch (mood) {
        MascotMood.happy => AppAssets.rizmaMascot,
        MascotMood.sleepy => AppAssets.rizmaMascotSleep,
        MascotMood.thinking => AppAssets.rizmaMascotThink,
      },
      width: size,
      height: size,
      colorMapper: _MascotColorMapper(
        body: color.body,
        dark: color.dark,
        leaf: color.fold,
      ),
    );
  }
}
```

ماذا يفعل؟ `build`: يختار ملف SVG حسب المزاج، بالحجم المطلوب، ومعه مبدِّل الألوان بالجسم والغامق والورقة من لون الطالب.

لماذا بهذه الطريقة؟ ملف متجه واحد لكل حالة يتحمّل كل الأحجام (34 حتى 170).

الأسطر 54–67

```
class _MascotColorMapper extends ColorMapper {
  const _MascotColorMapper({
    required this.body,
    required this.dark,
    required this.leaf,
  });

  static const _originalBody = Color(0xFF177A63);
  static const _originalDark = Color(0xFF0E5C4A);
  static const _originalLeaf = Color(0xFF7FD7BC);

  final Color body;
  final Color dark;
  final Color leaf;
```

ماذا يفعل؟ `_MascotColorMapper`: الألوان الأصلية الثلاثة في الملف، والألوان الجديدة.

لماذا بهذه الطريقة؟ الأصلية ثوابت حتى تُقارن بسرعة.

الأسطر 69–82

```
  @override
  Color substitute(
    String? id,
    String elementName,
    String attributeName,
    Color color,
  ) {
    // 35 نقارن بلا الشفافية: ظل الكعب نفسه لون الغامق بشفافية%.
    final opaque = color.withValues(alpha: 1);
    if (opaque == _originalBody) return body.withValues(alpha: color.a);
    if (opaque == _originalDark) return dark.withValues(alpha: color.a);
    if (opaque == _originalLeaf) return leaf.withValues(alpha: color.a);
    return color;
  }
```

ماذا يفعل؟ `substitute`: يقارن كل لون بلا شفافية؛ إن طابق أحد الثلاثة يعيد البديل بالشفافية الأصلية، وإلا يتركه.

لماذا بهذه الطريقة؟ ظلّ التميمة هو اللون الغامق بشفافية 35%، فالمقارنة بلا شفافية تلوّنه هو أيضًا.

الأسطر 84–93

```
  @override
  bool operator ==(Object other) =>
      other is _MascotColorMapper &&
      other.body == body &&
      other.dark == dark &&
      other.leaf == leaf;

  @override
  int get hashCode => Object.hash(body, dark, leaf);
}
```

ماذا يفعل؟ `==` و`hashCode` حسب الألوان الثلاثة.

لماذا بهذه الطريقة؟ `flutter_svg` يعيد رسم الصورة فقط إن تغيّر المبدِّل؛ المساواة الصحيحة تمنع رسمًا زائدًا.

lib/core/widgets/screen_top_bar.dart — زر الرجوع والشريط العلوي

lib/core/widgets/screen_top_bar.dart

الأسطر 1–10

```dart
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';

import '../constants/app_assets.dart';
import '../strings/ar.dart';
import '../theme/app_colors.dart';
import '../theme/app_text_styles.dart';
import 'ledge.dart';
import 'offline_chip.dart';
import 'pressable_scale.dart';
```

ماذا يفعل؟ الاستيرادات، ومنها `LedgeBox` و `OfflineChip`.

لماذا بهذه الطريقة؟ —

الأسطر 12–17

```dart
/// زر الرجوع المربّع (أبيض بحافة سفلية). أيقونته مرسومة بسهم لليمين =
/// اتجاه الرجوع في RTL.
class BackTile extends StatelessWidget {
  const BackTile({super.key, required this.onTap});

  final VoidCallback onTap;
```

ماذا يفعل؟ `BackTile` ومعامله الوحيد.

لماذا بهذه الطريقة؟ —

الأسطر 19–24

```dart
  @override
  Widget build(BuildContext context) {
    return Semantics(
      button: true,
      label: ArStrings.back,
      excludeSemantics: true,
```

ماذا يفعل؟ `Semantics`: زر باسم «رجوع».

لماذا بهذه الطريقة؟ الأيقونة وحدها لا تُقرأ لقارئ الشاشة.

الأسطر 25–32

```
      child: LedgeBox(
        radius: 14,
        depth: 3,
        clip: true,
        child: Material(
          color: AppColors.card,
          child: InkWell(
            onTap: onTap,
```

ماذا يفعل؟ `LedgeBox` أبيض بزوايا 14 وحافة 3 مقصوص، فيه `Material` و `InkWell`.

لماذا بهذه الطريقة؟ من التصميم الجديد: زر الرجوع أبيض ملموس (كان سماويًا مسطّحًا).

الأسطر 33–53

```
            child: SizedBox(
              width: 44,
              height: 44,
              child: Center(
                child: SvgPicture.asset(
                  AppAssets.back,
                  width: 22,
                  height: 22,
                  colorFilter: const ColorFilter.mode(
                    AppColors.ink,
                    BlendMode.srcIn,
                  ),
                ),
              ),
            ),
          ),
        ),
      ).pressable(),
    );
  }
}
```

ماذا يفعل؟ مربع 44 فيه سهم الرجوع مصبوغًا بالحبري، والإغلاق مع `.pressable`.

لماذا بهذه الطريقة؟ السهم مرسوم لليمين = الرجوع في RTL. 44 أصغر مساحة لمس مريحة.

الأسطر 55–73

```dart
/// الشريط العلوي: رجوع (يمين) ثم العنوان، وفي الطرف الآخر أو شارة [end].
/// قاعدة التصميم: الحالة تظهر مرة واحدة في الشريط العلوي (أو «دون إنترنت»).
class ScreenTopBar extends StatelessWidget {
  const ScreenTopBar({
    super.key,
    required this.title,
    this.subtitle,
    required this.onBack,
    this.end,
    this.showOffline = true,
  });

  final String title;
  final String? subtitle;
  final VoidCallback onBack;
  final Widget? end;

  /// إظهار شارة «دون إنترنت» حين لا يوجد [end].
  final bool showOffline;
```

ماذا يفعل؟ `ScreenTopBar` ومعاملاته: العنوان، الفرعي، الرجوع، عنصر الطرف الآخر، و`showOffline` (جديد).

لماذا بهذه الطريقة؟ قاعدة التصميم: «دون إنترنت» تظهر مرة واحدة في الشريط العلوي؛ الشاشة التي تضع عنصرًا آخر تتخلى عنها.

الأسطر 75–77

```dart
  @override
  Widget build(BuildContext context) {
    final trailing = end ?? (showOffline ? const OfflineChip() : null);
```

ماذا يفعل؟ العنصر الأخير: `end` إن مُرِّر، وإلا شارة «دون إنترنت» إن لم تُعطَّل.

لماذا بهذه الطريقة؟ —

الأسطر 79–100

```
    return Padding(
      padding: const EdgeInsetsDirectional.fromSTEB(16, 12, 16, 4),
      child: Row(
        spacing: 12,
        children: [
          BackTile(onTap: onBack),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                if (subtitle != null)
                  Text(subtitle!, style: AppTextStyles.caption13),
                Text(title, style: AppTextStyles.heading22),
              ],
            ),
          ),
          if (trailing != null) Flexible(child: trailing),
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ صف بهامش: الرجوع، ثم العنوان (والفرعي فوقه)، ثم العنصر الأخير داخل `Flexible`، والإغلاق.

لماذا بهذه الطريقة؟ `Flexible` يجعل الشارة تنكمش على الشاشات الضيقة بدل أن تدفع العنوان خارجها.

lib/core/widgets/rizma_switch.dart — مفتاح التشغيل

lib/core/widgets/rizma_switch.dart

الأسطر 1–6

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';
import '../utils/haptics.dart';

/// مفتاح التشغيل بحسب التصميم (32×52). المقبض يبدأ من اليمين وهو مطفأ (RTL).
```

ماذا يفعل؟ استيراد Material والألوان وتعليق الصنف.

لماذا بهذه الطريقة؟ التصميم 32×52؛ والمقبض يبدأ من اليمين وهو مطفأ (RTL).

الأسطر 7–13

```
class RizmaSwitch extends StatelessWidget {
  const RizmaSwitch({
    super.key,
    required this.value,
    required this.onChanged,
    required this.semanticsLabel,
  });
```

ماذا يفعل؟ صنف `RizmaSwitch` وباني بثلاثة معاملات مطلوبة.

لماذا بهذه الطريقة؟ القيمة والدالة ووصف قارئ الشاشة.

الأسطر 15–17

```
  final bool value;
  final ValueChanged<bool> onChanged;
  final String semanticsLabel;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ الودجت بلا حالة؛ المالك يحتفظ بالقيمة.

الأسطر 19–22

```
  void _toggle() {
    Haptics.tap();
    onChanged(!value);
  }
```

ماذا يفعل؟ `_toggle`: اهتزاز خفيف ثم استدعاء `onChanged` بالقيمة المعكوسة.

لماذا بهذه الطريقة؟ نقطة واحدة يستعملها الضغط والوصول (accessibility) معًا فلا يختلف سلوكهما، وإضافة الاهتزاز في مكان واحد.

الأسطر 24–25

```
  @override
  Widget build(BuildContext context) {
```

ماذا يفعل؟ `build` .

لماذا بهذه الطريقة؟ يعيد المفتاح.

الأسطر 26–30

```
    return Semantics(
      toggled: value,
      label: semanticsLabel,
      excludeSemantics: true,
      onTap: _toggle,
```

ماذا يفعل؟ `Semantics` تعلن مفتاحًا بحالته (`toggled`) واسمه وتستجيب للمس.

لماذا بهذه الطريقة؟ ضروري لقارئ الشاشة لأن المفتاح مرسوم يدويًا.

الأسطر 31–33

```
      child: GestureDetector(
        onTap: _toggle,
        behavior: HitTestBehavior.opaque,
```

ماذا يفعل؟ `GestureDetector` يقلب القيمة عند الضغط.

لماذا بهذه الطريقة؟ `HitTestBehavior.opaque` يجعل كل المساحة قابلة للضغط حتى الشفافة.

الأسطر 34–38

```
        child: AnimatedContainer(
          duration: const Duration(milliseconds: 150),
          width: 52,
          height: 32,
          padding: const EdgeInsets.all(4),
```

ماذا يفعل؟ `AnimatedContainer` بحجم 52×32 وحشوة 4.

لماذا بهذه الطريقة؟ الانتقال ناعم 150 مللي ثانية.

الأسطر 39–42

```
          decoration: BoxDecoration(
            color: value ? AppColors.teal : AppColors.disabled,
            borderRadius: BorderRadius.circular(16),
          ),
```

ماذا يفعل؟ لون المسار: teal إن مفعّل وإلا `disabled`، وزوايا 16.

لماذا بهذه الطريقة؟ مطابق لـ Figma.

الأسطر 43–47

```
          child: AnimatedAlign(
            duration: const Duration(milliseconds: 150),
            alignment: value
                ? AlignmentDirectional.centerEnd
                : AlignmentDirectional.centerStart,
```

ماذا يفعل؟ `AnimatedAlign`: المقبض عند `centerEnd` إن مفعّل وإلا `centerStart`.

لماذا بهذه الطريقة؟ بصيغة `Directional` فيبدأ من اليمين وهو مطفأ في RTL.

الأسطر 48–54

```
            child: const DecoratedBox(
              decoration: BoxDecoration(
                color: AppColors.card,
                shape: BoxShape.circle,
              ),
              child: SizedBox(width: 24, height: 24),
            ),
```

ماذا يفعل؟ المقبض دائرة بيضاء 24×24.

لماذا بهذه الطريقة؟ من Figma.

الأسطر 55–60

```
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/core/widgets/dashed_border.dart — الحد المنقّط

lib/core/widgets/dashed_border.dart

الأسطر 1–4

```
import 'package:flutter/material.dart';

/// صندوق بحد منقّط وزوايا مدوّرة (الصناديق الفارغة في التصميم).
class DashedBorderBox extends StatelessWidget {
```

ماذا يفعل؟ استيراد Material وتعليق وصنف `DashedBorderBox`.

لماذا بهذه الطريقة؟ تُستعمل في «طالب جديد» وصندوق المراجعة الفارغ.

الأسطر 5–14

```
  const DashedBorderBox({
    super.key,
    required this.child,
    required this.color,
    this.radius = 18,
    this.strokeWidth = 2,
    this.dash = 6,
    this.gap = 5,
    this.padding = EdgeInsets.zero,
  });
```

ماذا يفعل؟ الباني: المحتوى واللون مطلوبان، والباقي اختياري (زوايا 18، سُمك 2، شرطة 6، فراغ 5، حشوة صفر).

لماذا بهذه الطريقة؟ قابل للتخصيص: «طالب جديد» يستعمل شرطة 8 وفراغ 6.

الأسطر 16–22

```
  final Widget child;
  final Color color;
  final double radius;
  final double strokeWidth;
  final double dash;
  final double gap;
  final EdgeInsetsGeometry padding;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ تُمرَّر للرسّام.

الأسطر 24–26

```
24    @override
25    Widget build(BuildContext context) {
26      return CustomPaint(
```

ماذا يفعل؟ `build` يعيد `CustomPaint`.

لماذا بهذه الطريقة؟ لا يوجد في Flutter حد منقّط جاهز لمستطيل مدوّر، فنرسمه.

الأسطر 27–33

```
27        painter: _DashedRRectPainter(
28          color: color,
29          radius: radius,
30          strokeWidth: strokeWidth,
31          dash: dash,
32          gap: gap,
33        ),
```

ماذا يفعل؟ نمرّر للرسّام اللون والزوايا والسُمك والشرطة والفراغ.

لماذا بهذه الطريقة؟ رسّام خاص بالملف.

الأسطر 34–37

```
34        child: Padding(padding: padding, child: child),
35      );
36    }
37  }
```

ماذا يفعل؟ الابن داخل `Padding` ثم إغلاق الودجت.

لماذا بهذه الطريقة؟ المحتوى يظهر فوق الحدّ المرسوم.

الأسطر 39–46

```
39  class _DashedRRectPainter extends CustomPainter {
40    _DashedRRectPainter({
41      required this.color,
42      required this.radius,
43      required this.strokeWidth,
44      required this.dash,
45      required this.gap,
46    });
```

ماذا يفعل؟ الصنف الخاص `_DashedRRectPainter` وبانيه.

لماذا بهذه الطريقة؟ `CustomPainter` هو الصنف الذي يرسم على اللوحة.

الأسطر 52–48

```
48    final Color color;
49    final double radius;
50    final double strokeWidth;
51    final double dash;
52    final double gap;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 55–54

```
54    @override
55    void paint(Canvas canvas, Size size) {
```

ماذا يفعل؟ دالة `paint(canvas, size)`.

لماذا بهذه الطريقة؟ تُستدعى عند الرسم.

الأسطر 62–56

```
56      final inset = strokeWidth / 2;
57      final rect = Rect.fromLTWH(
58        inset,
59        inset,
60        size.width - strokeWidth,
61        size.height - strokeWidth,
62      );
```

ماذا يفعل؟ نحسب مستطيلًا مزاحًا بنصف سُمك الخط من كل جهة.

لماذا بهذه الطريقة؟ كي لا يُقصّ نصف الخط عند حواف الودجت.

الأسطر 64–63

```
63      final path = Path()
64        ..addRRect(RRect.fromRectAndRadius(rect, Radius.circular(radius)));
```

ماذا يفعل؟ نبني مسارًا بمستطيل مدوّر.

لماذا بهذه الطريقة؟ `RRect` = مستطيل بزوايا مدوّرة.

الأسطر 68–65

```
65      final paint = Paint()
66        ..color = color
67        ..style = PaintingStyle.stroke
68        ..strokeWidth = strokeWidth;
```

ماذا يفعل؟ فرشاة بلون الحد ونمط «خط» وسُمك محدد.

لماذا بهذه الطريقة؟ `PaintingStyle.stroke` يرسم الحدود لا التعبئة.

الأسطر 70–77

```
70      for (final metric in path.computeMetrics()) {
71        var distance = 0.0;
72        while (distance < metric.length) {
73          final end = (distance + dash).clamp(0, metric.length).toDouble();
74          canvas.drawPath(metric.extractPath(distance, end), paint);
75          distance += dash + gap;
76        }
77      }
```

ماذا يفعل؟ نمرّ على كل مقطع من المسار ونرسم شرطات بطول `dash` يفصل بينها `gap`.

لماذا بهذه الطريقة؟ `computeMetrics` تعطينا طول المسار لنقتطع منه شرطات متتالية. `clamp` يمنع تجاوز نهاية المسار.

السطر 78

```
78    }
```

ماذا يفعل؟ إغلاق `paint`.

لماذا بهذه الطريقة؟ نهاية الدالة.

الأسطر 80–87

```
80    @override
81    bool shouldRepaint(_DashedRRectPainter old) =>
82        old.color != color ||
83        old.radius != radius ||
84        old.strokeWidth != strokeWidth ||
85        old.dash != dash ||
86        old.gap != gap;
87  }
```

ماذا يفعل؟ `shouldRepaint`: نعيد الرسم فقط إن تغيّرت إحدى القيم، ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ تحسين أداء: لا نرسم من جديد بلا داعٍ.

lib/core/widgets/pin_pad.dart — نقاط الرمز السري ولوحة الأرقام

`lib/core/widgets/pin_pad.dart`

السلاسة: كل مفتاح (`_PinKey`) يُلحَق بـ `.pressable(enabled: onTap != null)` ، وضغط الرقم أو الحذف يهتزّ خفيفًا. اللوحة كلها داخل `FittedBox(scaleDown)` لتتقلّص على الشاشات الضيقة.

الأسطر 1–8

```dart
import 'package:flutter/material.dart';

import '../strings/ar.dart';
import '../theme/app_colors.dart';
import 'app_icon.dart';
import 'ledge.dart';
import 'pressable_scale.dart';
import '../utils/haptics.dart';
```

ماذا يفعل؟ استيراد Material والنصوص والألوان.

لماذا بهذه الطريقة؟ ما تحتاجه النقاط واللوحة.

الأسطر 10–17

```dart
/// نقاط الرمز السري: المملوء منها = أرقام أُدخلت.
class PinDots extends StatelessWidget {
  const PinDots({
    super.key,
    required this.length,
    required this.filled,
    this.error = false,
  });
```

ماذا يفعل؟ تعليق وصنف `PinDots` وباني: `length` و`filled` مطلوبان و`error` افتراضيه false.

لماذا بهذه الطريقة؟ النقاط تعرض عدد الأرقام المُدخلة دون كشفها.

الأسطر 19–21

```dart
  final int length;
  final int filled;
  final bool error;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ الطول والمُدخَل والخطأ.

الأسطر 23–24

```dart
  @override
  Widget build(BuildContext context) {
```

ماذا يفعل؟ `build`.

لماذا بهذه الطريقة؟ يعيد النقاط.

الأسطر 25–27

```dart
    return Semantics(
      label: ArStrings.pinDotsSemantics(filled, length),
      excludeSemantics: true,
```

ماذا يفعل؟ `Semantics` تقرأ «أُدخل N من 4 أرقام».

لماذا بهذه الطريقة؟ لا نكشف الأرقام لقارئ الشاشة.

الأسطر 28–33

```
28        child: Padding(
29          padding: const EdgeInsets.symmetric(vertical: 10),
30          child: Row(
31            mainAxisSize: MainAxisSize.min,
32            spacing: 18,
33            children: [
```

ماذا يفعل؟ حشوة عمودية 10 و`Row` بحجم أدنى ومسافة 18.

لماذا بهذه الطريقة؟ مسافة التصميم بين النقاط.

الأسطر 34–38

```
34              for (var i = 0; i < length; i++)
35                AnimatedContainer(
36                  duration: const Duration(milliseconds: 120),
37                  width: 22,
38                  height: 22,
```

ماذا يفعل؟ حلقة تبني نقطة لكل رقم بـ `AnimatedContainer` (120 مللي ثانية)، 22×22.

لماذا بهذه الطريقة؟ الامتلاء يظهر بحركة قصيرة ناعمة. كبرت النقاط قليلًا مع التصميم الجديد.

الأسطر 39–40

```
39                  decoration: BoxDecoration(
40                    shape: BoxShape.circle,
```

ماذا يفعل؟ حجم 20×20 وشكل دائري.

لماذا بهذه الطريقة؟ من Figma.

الأسطر 41–43

```
41                    color: i < filled
42                        ? (error ? AppColors.coral : AppColors.teal)
43                        : Colors.transparent,
```

ماذا يفعل؟ النقطة المملوءة: teal (أو مرجاني عند الخطأ)، وغير المملوءة شفافة.

لماذا بهذه الطريقة؟ اللون المرجاني يدل على إدخال خاطئ.

الأسطر 44–51

```
44                    border: i < filled
45                        ? null
46                        : Border.all(
47                            color: error ? AppColors.coral : AppColors.disabled,
48                            width: 2,
49                          ),
50                  ),
51                ),
```

ماذا يفعل؟ الحد: لا حد للمملوءة، وحد 2 رمادي (أو مرجاني) للفارغة.

لماذا بهذه الطريقة؟ الفارغة دائرة بحد، والمملوءة دائرة مصمتة.

الأسطر 52–60

```
          ],
        ),
      ),
    );
  }
}

/// لوحة أرقام الرمز السري (1 2 3 / 4 5 6 / 7 8 9 / فراغ 0 حذف) باتجاه الواجهة
/// في RTL يظهر «1» يمينًا والحذف يسارًا، كما في التصميم الجديد.
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية `PinDots`.

السطر 61

```
class PinPad extends StatelessWidget {
```

ماذا يفعل؟ تعليق وصنف `PinPad`: الترتيب 1 2 3 / 4 5 6 / 7 8 9 / فراغ 0 حذف، وفي RTL يظهر «1» يمينًا والحذف يسارًا.

لماذا بهذه الطريقة؟ تغيّر في التصميم الجديد: كانت اللوحة مبنية LTR عمدًا (1 يسارًا)؛ التصميم الجديد يتبع اتجاه الواجهة العربية، فأُزيل `Directionality(ltr)`.

الأسطر 62–67

```
  const PinPad({
    super.key,
    required this.onDigit,
    required this.onDelete,
    this.enabled = true,
  });
```

ماذا يفعل؟ الباني: دالة الرقم ودالة الحذف مطلوبتان و`enabled` افتراضيه true.

لماذا بهذه الطريقة؟ `enabled: false` أثناء القفل.

الأسطر 69–71

```
  final ValueChanged<String> onDigit;
  final VoidCallback onDelete;
  final bool enabled;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ الدوال والحالة.

الأسطر 73–78

```
73    static const _rows = [
74      ['1', '2', '3'],
75      ['4', '5', '6'],
76      ['7', '8', '9'],
77      ['', '0', 'del'],
78    ];
```

ماذا يفعل؟ ثابت `_rows`: أرقام اللوحة؛ الفراغ نص فارغ والحذف `del`.

لماذا بهذه الطريقة؟ نبني اللوحة من بيانات لا من 12 ودجت يدويًا.

الأسطر 80–81

```
80    @override
81    Widget build(BuildContext context) {
```

ماذا يفعل؟ `build` (توقيع الدالة).

لماذا بهذه الطريقة؟ تعيد اللوحة كلها.

الأسطر 82–86

```
82      // FittedBox: على شاشة أضيق من 276 (عرض اللوحة) تتقلّص اللوحة بدل أن تفيض.
83      return FittedBox(
84        fit: BoxFit.scaleDown,
85        // اتجاه الواجهة (RTL): «1» في اليمين والحذف على اليسار كما في
86        // التصميم الجديد (التصميم القديم كانت اللوحة LTR في).
```

ماذا يفعل؟ تعليق ثم `FittedBox(fit: scaleDown)` يلفّ اللوحة، وتعليق أن اللوحة تتبع اتجاه الواجهة (RTL).

لماذا بهذه الطريقة؟ عرض اللوحة الطبيعي 300 (3 مفاتيح × 88 + مسافتان 18). على شاشة أضيق تتقلّص اللوحة كلها بدل أن تخرج عن الحدود.

الأسطر 87–90

```
87        child: Column(
88          mainAxisSize: MainAxisSize.min,
89          spacing: 12,
90          children: [
```

ماذا يفعل؟ `Column` بحجم أدنى ومسافة 12 بين الصفوف وبداية `children`.

لماذا بهذه الطريقة؟ مسافة Figma.

الأسطر 91–96

```
91            for (final row in _rows)
92              Row(
93                mainAxisSize: MainAxisSize.min,
94                spacing: 18,
95                children: [for (final key in row) _buildKey(key)],
96              ),
```

ماذا يفعل؟ لكل صف `Row` بمسافة 18 يضم مفاتيحه.

لماذا بهذه الطريقة؟ نبني الصفوف من بيانات `_rows`.

الأسطر 97–100

```
97          ],
98        ),
99      );
100   }
```

ماذا يفعل؟ إغلاق `Column` و`FittedBox` و`build`.

لماذا بهذه الطريقة؟ نهاية الدالة.

الأسطر 102–103

```
102   Widget _buildKey(String key) {
103     if (key.isEmpty) return const SizedBox(width: 88, height: 63);
```

ماذا يفعل؟ `_buildKey`: المفتاح الفارغ مساحة 88×63 بلا رسم.

لماذا بهذه الطريقة؟ يحفظ مكان الخانة الفارغة بمقاس المفتاح مع حافته (60 + 3).

الأسطر 104–116

```
104     if (key == 'del') {
105       return _PinKey(
106         semanticsLabel: ArStrings.pinDelete,
107         onTap: enabled
108             ? () {
109                 Haptics.tap();
110                 onDelete();
111               }
112             : null,
113         filled: false,
114         child: const AppIcon('del', size: 26, color: AppColors.ink),
115       );
116     }
```

ماذا يفعل؟ مفتاح الحذف: اسم «حذف» وأيقونة `del` من مجموعة التصميم؛ ويتعطل إن كانت اللوحة معطّلة.

لماذا بهذه الطريقة؟ `onTap: null` يعطّله. أيقونة التصميم بدل أيقونة Material.

الأسطر 117–125

```
    return _PinKey(
      semanticsLabel: key,
      onTap: enabled
          ? () {
              Haptics.tap();
              onDigit(key);
            }
          : null,
      filled: true,
```

ماذا يفعل؟ مفتاح رقم: اسمه الرقم نفسه وينادي `onDigit` وخلفيته بيضاء.

لماذا بهذه الطريقة؟ قارئ الشاشة يقرأ الرقم.

الأسطر 126–137

```
      child: Text(
        key,
        style: const TextStyle(
          fontFamily: 'Cairo',
          fontSize: 28,
          fontWeight: FontWeight.w700,
          height: 1.2,
          color: AppColors.ink,
        ),
      ),
    );
  }
```

ماذا يفعل؟ نص الرقم بنمط يدوي: Cairo حجم 28 وزن 700.

لماذا بهذه الطريقة؟ نمط خاص بالمفاتيح ولا يتكرر في مكان آخر فلا نضيفه لـ `AppTextStyles`.

السطر 138

```
}
```

ماذا يفعل؟ إغلاق `PinPad`.

لماذا بهذه الطريقة؟ نهاية الصنف.

الأسطر 140–146

```
class _PinKey extends StatelessWidget {
  const _PinKey({
    required this.semanticsLabel,
    required this.onTap,
    required this.filled,
    required this.child,
  });
```

ماذا يفعل؟ صنف خاص `_PinKey` وباني بأربعة معاملات.

لماذا بهذه الطريقة؟ مفتاح واحد من اللوحة.

الأسطر 148–151

```
148   final String semanticsLabel;
149   final VoidCallback? onTap;
150   final bool filled;
151   final Widget child;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ الاسم والدالة وهل له خلفية والمحتوى.

الأسطر 153–165

```
153   @override
154   Widget build(BuildContext context) {
155     return Semantics(
156       button: true,
157       enabled: onTap != null,
158       label: semanticsLabel,
159       excludeSemantics: true,
160       // مفتاح ملموس (التصميم): أبيض بحافة سفلية، ومفتاح الحذف سماوي.
161       child: LedgeBox(
162         color: filled ? AppColors.card : AppColors.sky,
163         radius: 16,
164         depth: 3,
165         clip: true,
```

ماذا يفعل؟ `build` و`Semantics` لزر بحالة تفعيله واسمه، ثم `LedgeBox` بزوايا 16 وحافة 3 (أبيض للأرقام، سماوي للحذف) مقصوص.

لماذا بهذه الطريقة؟ التصميم الجديد: مفاتيح مستطيلة ملموسة بدل الكبسولات ذات الحدّ.

الأسطر 166–167

```
166         child: Material(
167           color: filled ? AppColors.card : AppColors.sky,
```

ماذا يفعل؟ `Material` بلون المفتاح نفسه.

لماذا بهذه الطريقة؟ ليظهر تموّج `InkWell` فوق اللون.

الأسطر 168–176

```
168           child: InkWell(
169             onTap: onTap,
170             child: SizedBox(width: 88, height: 60, child: Center(child: child)),
171           ),
172         ),
173       ),
174     ).pressable(enabled: onTap != null);
175   }
176 }
```

ماذا يفعل؟ `InkWell` وصندوق 88×60 بمحتوى في المنتصف، ثم الإغلاق مع `.pressable`.

لماذا بهذه الطريقة؟ مقاس المفتاح من التصميم الجديد (أعرض وأقصر قليلًا).